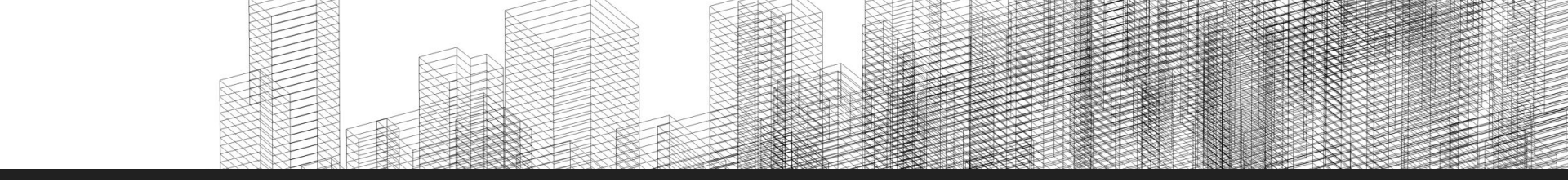


Introduction to Embedded Device Reverse Engineering

From electrical pins to root shells.

TRAINER INTRODUCTION

Pedro Ribeiro (@pedrib1337)

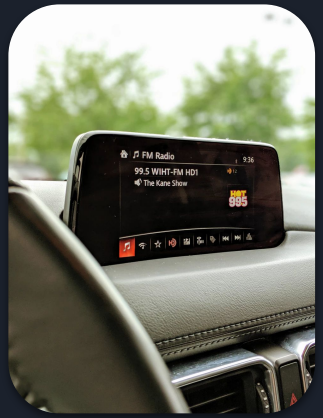
- Career path: ISO27001 audit -> pentesting -> pwning
- Hits anything that can get RCE'ed (Java Enterprise Apps, IoT, SCADA, Baseband)
- Founder & Director of Research at Agile Information Security
- 180+ public vulns with CVE (mostly RCE) and 60+ Metasploit modules authore
- Wannabe YouTuber with Radek Domanski as the FlashbackTeam (https://www.youtube.com/@FlashbackTeam)

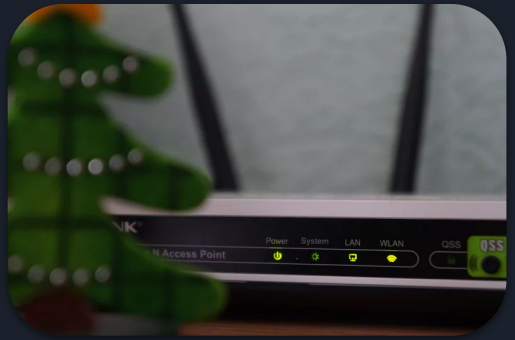
IMPORTANT NOTE

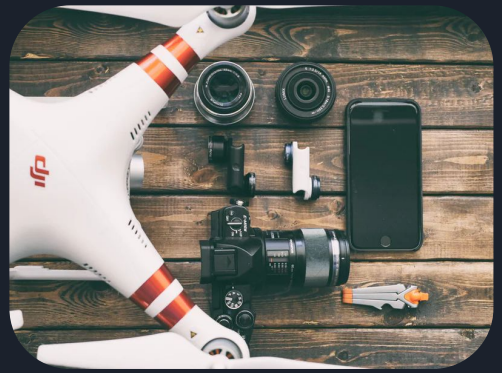
This is a HIGHLY SIMPLIFIED and much shorter version of the training course “Hunting Zero-Days in Embedded Devices” (HZED), which I (Pedro Ribeiro) teach with my colleague Radek Domanski.

While this smaller version was made with much care and love, it does not represent the same quality of the HZED course, which is a massively comprehensive journey teaching you how to analyse the hardware, extract the firmware, find vulnerabilities and exploit them, on all kinds of embedded devices.

EMBEDDED DEVICES







Instruction Set Architectures

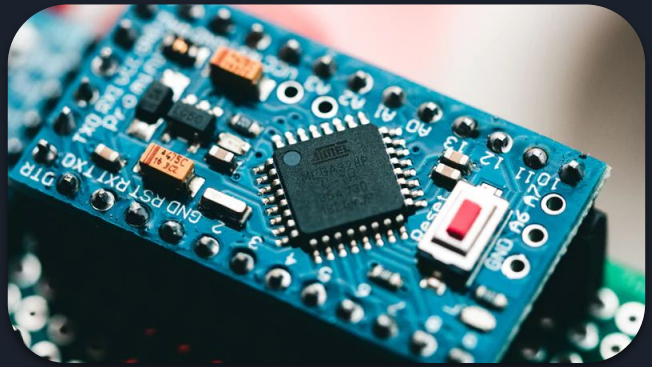
Common:

- ARM
- MIPS
- X86

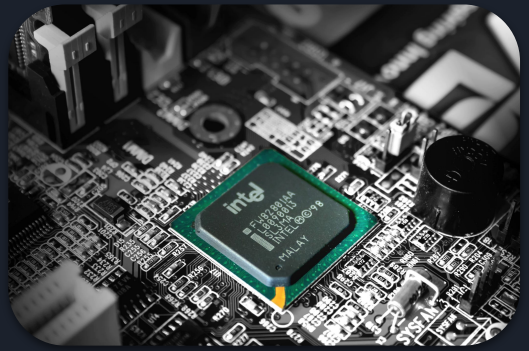
Exotic:

- PIC
- AVR
- Renesas V850
- PowerPC
- SPARC

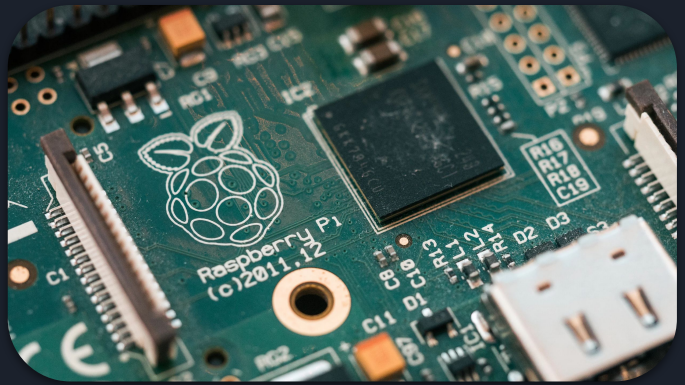
Microcontroller



Microprocessor



System-on-a-chip (SOC)



FPGA

Bare-Metal

- An application that runs on a microcontroller without Operating System
- It uses features provided by the MCU, i.e. peripherals I/O to sensors
- It runs a single application / process. There is no scheduler or context switching
- Very small memory footprint and very fast boot / reaction times
- Example: A thermostat application that continuously reads values from attached sensor and controls a heater via another attached I/O

Real-Time Operating System (RTOS)

- Minimalistic OS with a guaranteed real-time execution
- Small memory footprint and very fast boot time (miliseconds vs seconds)
- Processes are implemented in a so-called tasks.
- Higher priority tasks are guaranteed to execute completely before lower priority ones. When RTOS received an interrupt with a higher priority task, a scheduler will immediately pause current execution and switch to requested task.
- RTOS implements supporting functions, i.e. IP Stack, CAN stack, etc
- Example: Vehicle AirBag ECU, when a crash occurs, a RTOS has to guarantee to execute a task responsible for airbag deployment in a predefined time-frame. Not too early and not too late. Just In time.

Embedded Linux

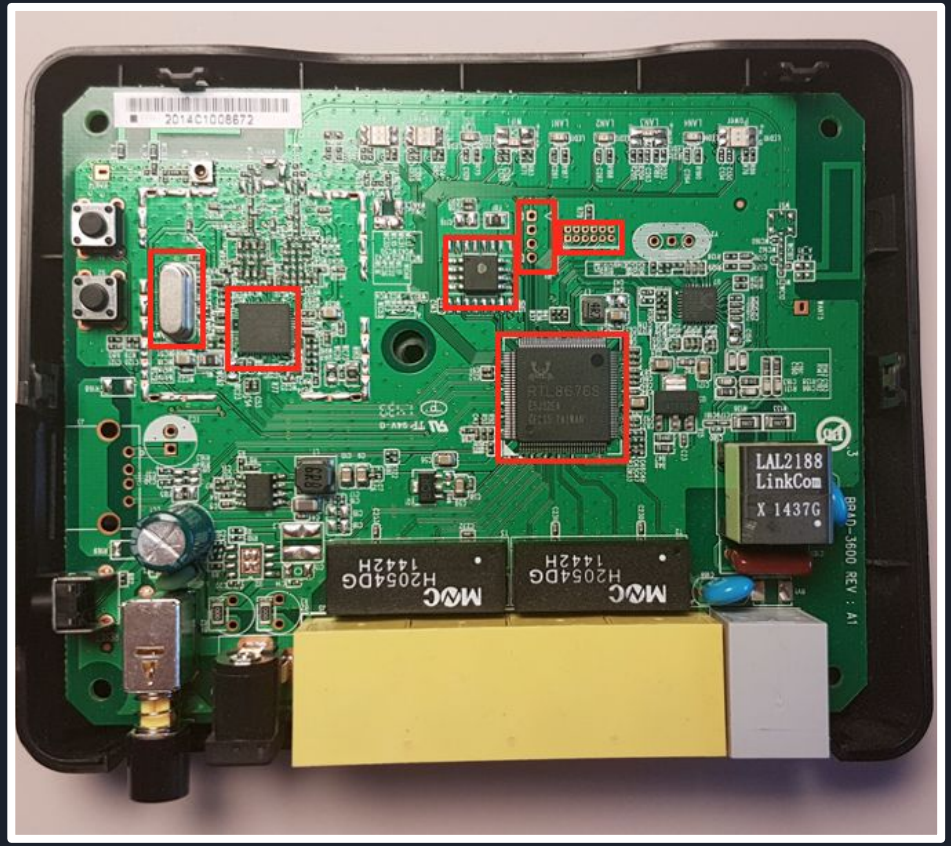
- Stripped down kernel
- Implements process separation, memory management, network communication, etc
- Typical Linux scheduler: fair amount of CPU assigned to all running processes
- Much smaller size than a typical Linux OS, but stil not comparable with RTOS or Bare-Metal

What is hardware hacking?

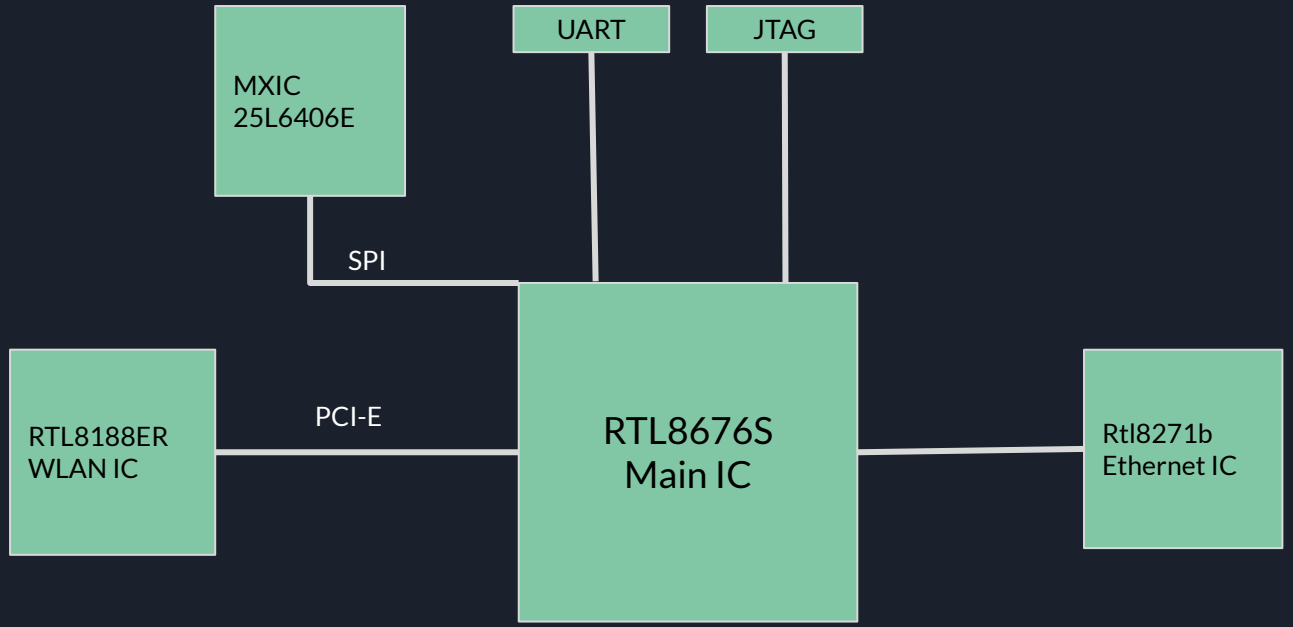
- Locating debug interfaces
- Dumping firmware
- Glitching
- PCB/Hardware reverse engineering
- Bypassing security restrictions by modifying hardware
- Hardware implants

Inside a Router

- Try to identify components and communication lines
- Map architecture / design



Inside a Router - Architecture

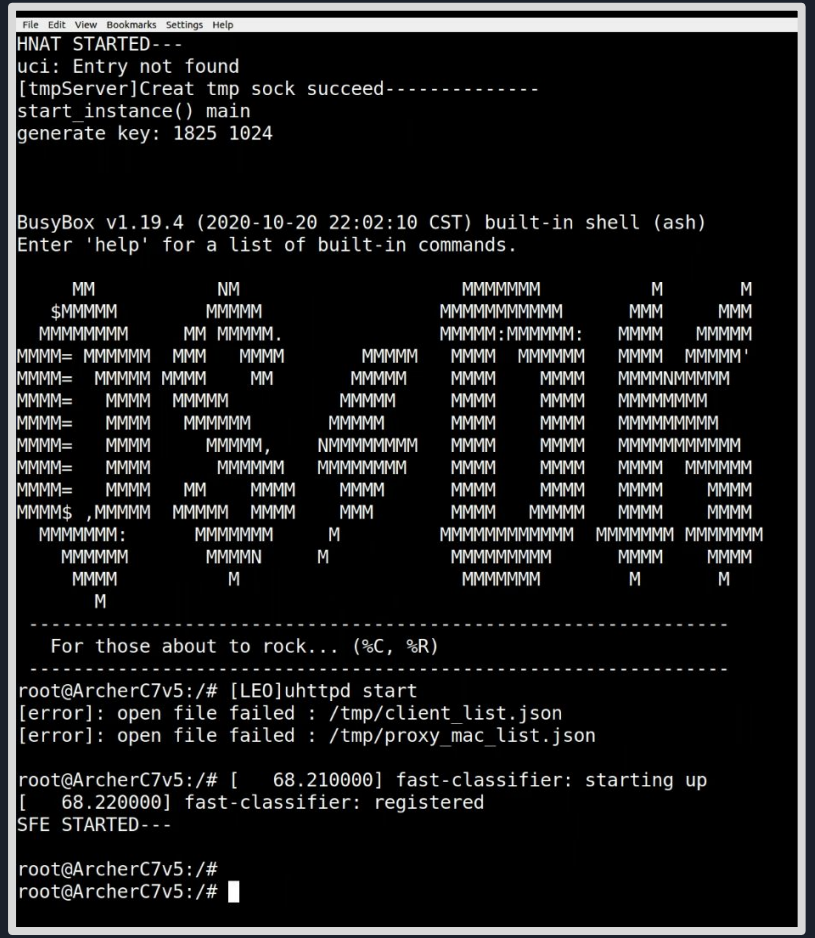


Exercise
Hardware components

- UART stands for Universal Asynchronous Receiver-Transmitter
- It is a device used for asynchronous serial communication between two parties
  - Such as your computer and an embedded / IoT device!
  - Or different components in the same board, device, etc
- Data format and transmission speeds are configurable, but there are some widely used defaults
- Almost all embedded / IoT devices, and even other devices like computers, have a UART device for communicating with the outside world
- **So what? Why do we care about this?**

What can we do with UART?

- Most embedded / IoT devices have a “serial console” which is used for various purposes during manufacturing, debugging, etc.
- If we’re lucky, when we connect to it over UART, we get a root shell!
  - Which will help us do recon on the device, debug exploits, etc
  - Get the device’s firmware / filesystem contents
  - Attack the bootloader, access encrypted memory, etc



It's not that easy though…

- Some device manufacturers disable the serial console before shipping a device
- A secret key combination is required to activate the login shell
- The login password might be randomised and specific to each device or even each boot
- The shell might be restricted and only allow us to run certain commands
- Read only console, no commands can be entered

It's still useful for hackers like us

- Use command injection or other trickery to bypass the shell restrictions
- Find out how the login password is created and attempt to crack it
- We can dump firmware over serial console restricted shell :D
- Interrupt the bootloader and use tricks to get a root shell
- Even if it's read only it's still useful
    - Boot logs showing hardware specs
    - Encryption keys and passwords!
    - Crash logs from our exploitation attempts!

How to connect to UART

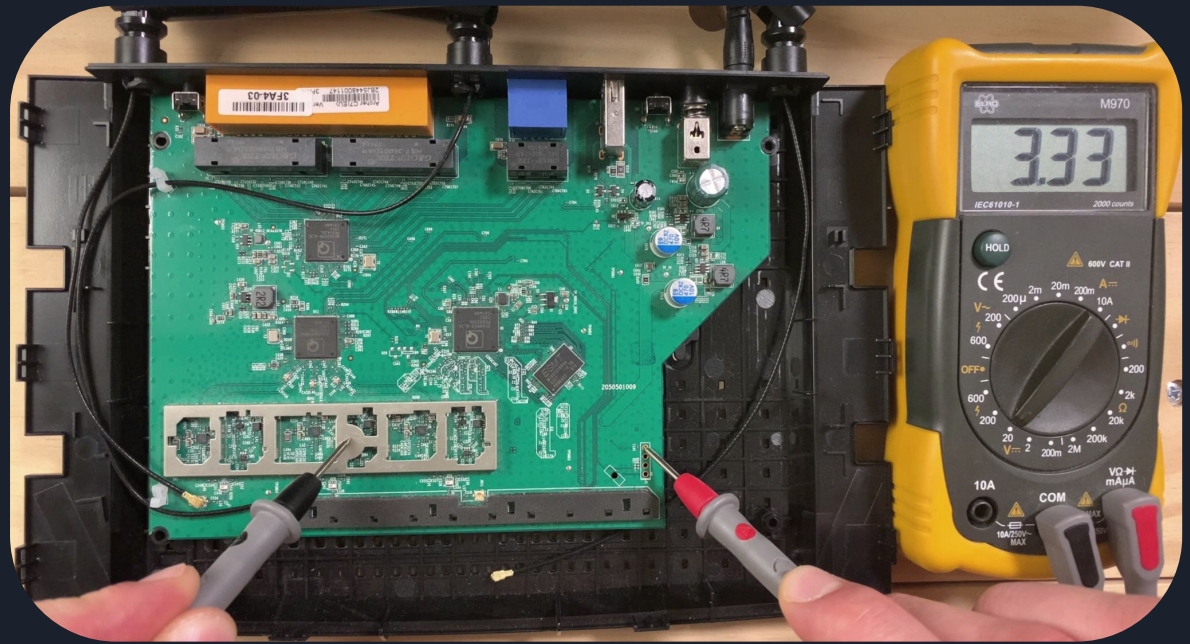
- Find UART pins
- Identify baud rate
- Use a UART capable microcontroller to connect to a target
- Setup a UART connection
- Enjoy!

How to identify a function of each pin
GND

- Use a multimeter
  - Target powered-off
  - Turn on continuity mode
  - One test lead on grounded material
  - Other one inspects each pin until you hear a *beep*

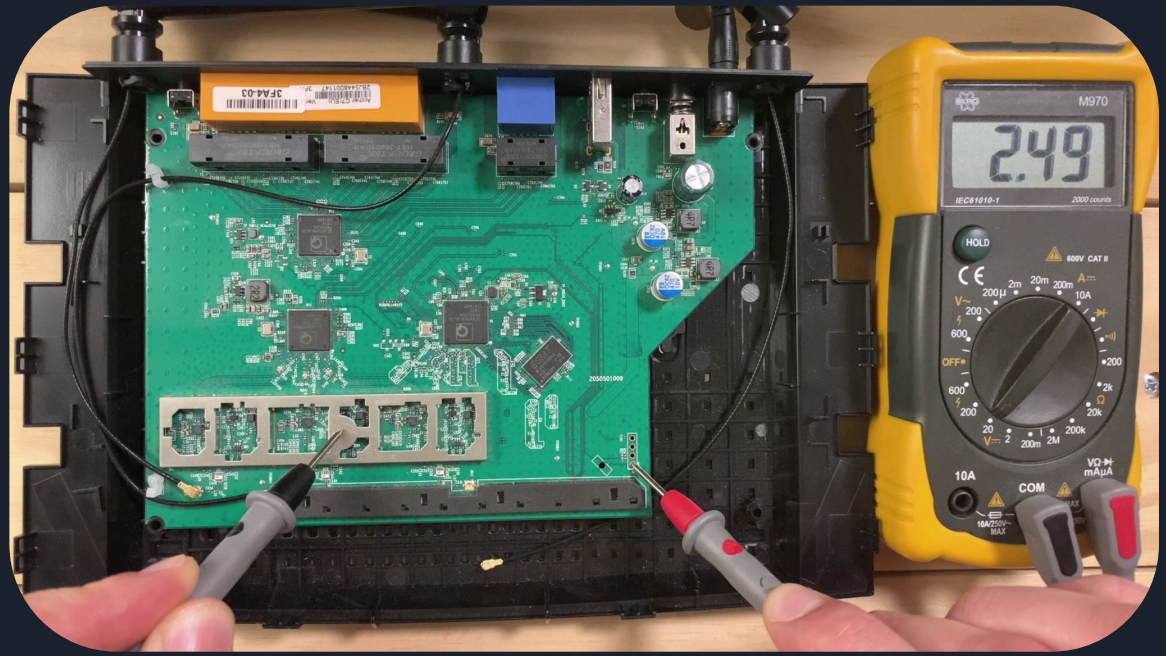
How to identify a function of each pin
VCC

- Use a multimeter (Voltage setting)
  - Target powered-on
  - Black lead on GND
  - Measure voltage of each pin
  - VCC most likely will have a constant voltage, i.e. 3.3V



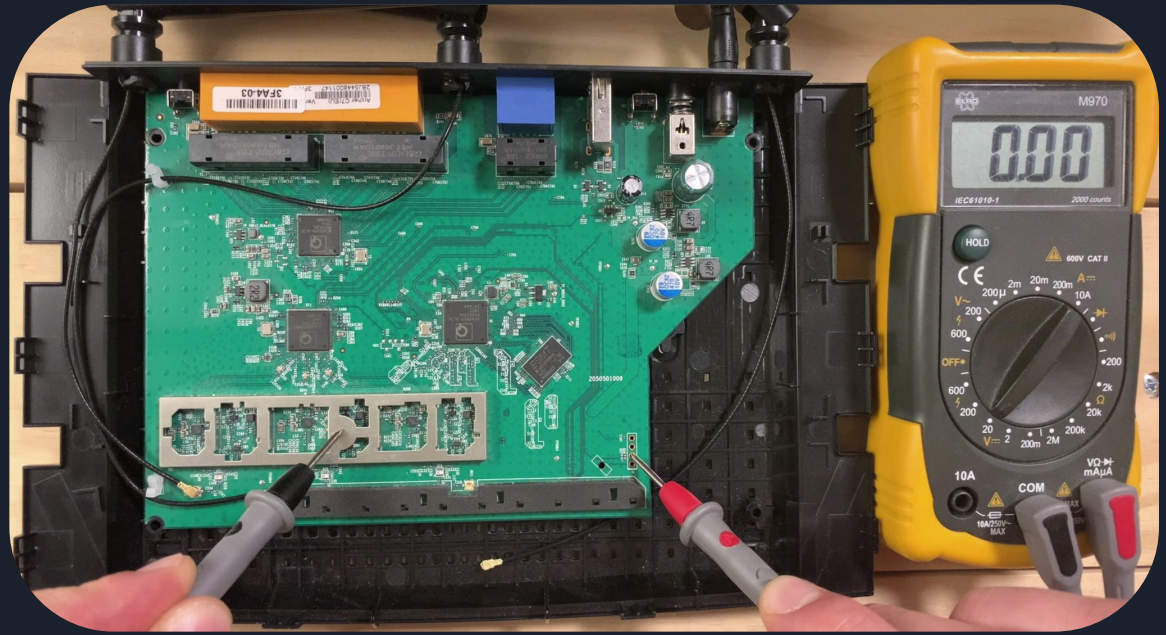
How to identify a function of each pin
Tx

- Use a multimeter (Voltage setting)
  - Target powered-on
  - Black lead on GND
  - Measure voltage of each pin
  - If Tx is active, in most of the cases, it will have a variable voltage - it's sending data



How to identify a function of each pin
Rx

- Use a multimeter (Voltage setting)
  - Target powered-on
  - Black lead on GND
  - Measure voltage of each pin
  - Rx is tricky, it can be pulled-up, pulled-down or fluctuate.

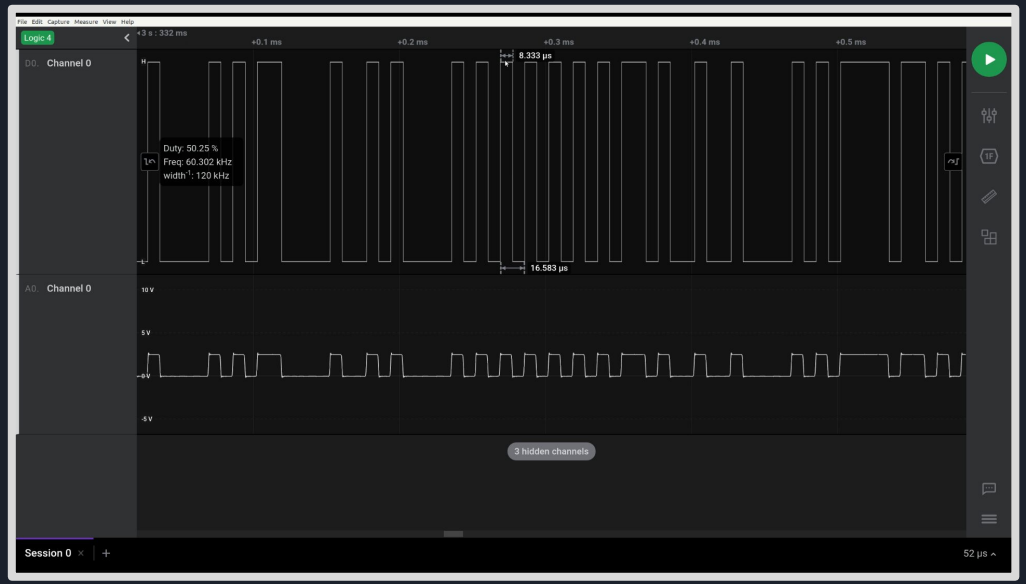


How to connect to UART

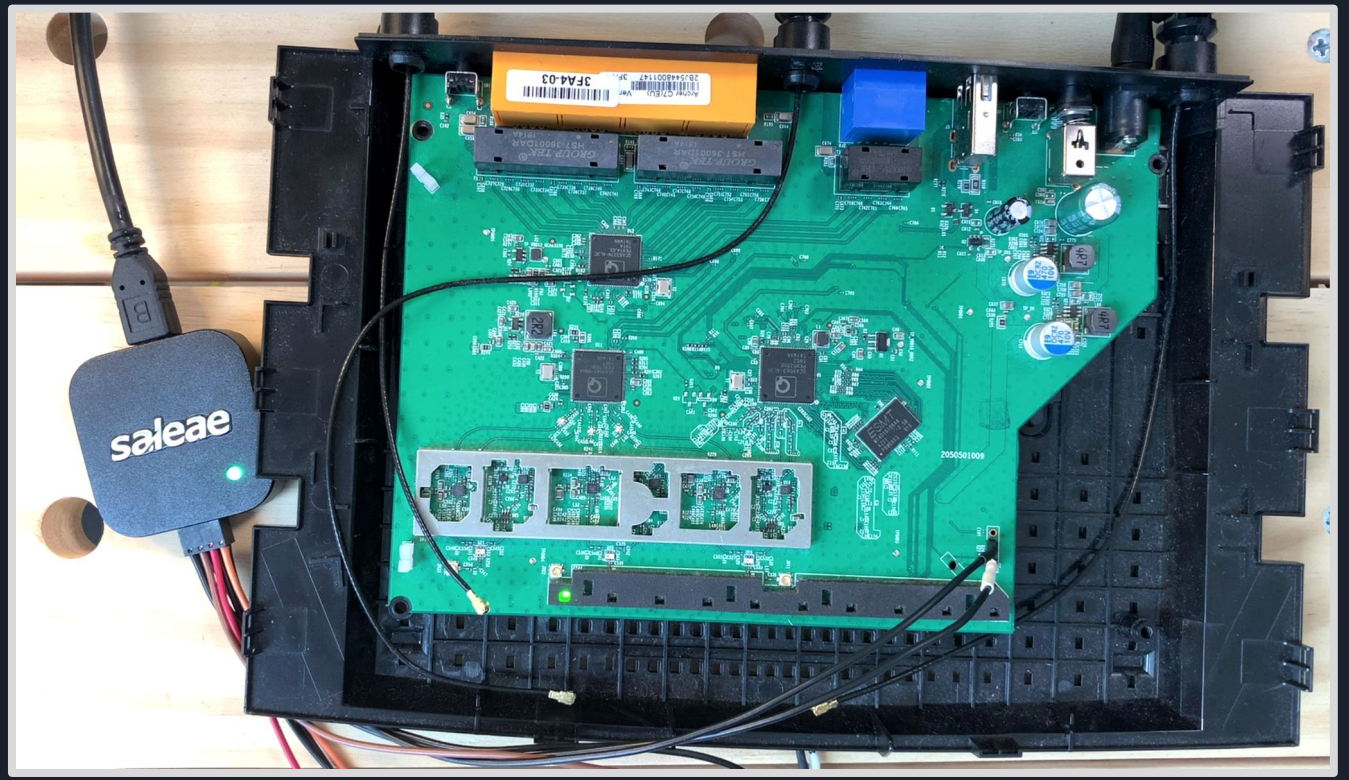
- Find UART pins
- **Identify baud rate**
- Use a UART capable microcontroller to connect to a target
- Setup a UART connection
- Enjoy!

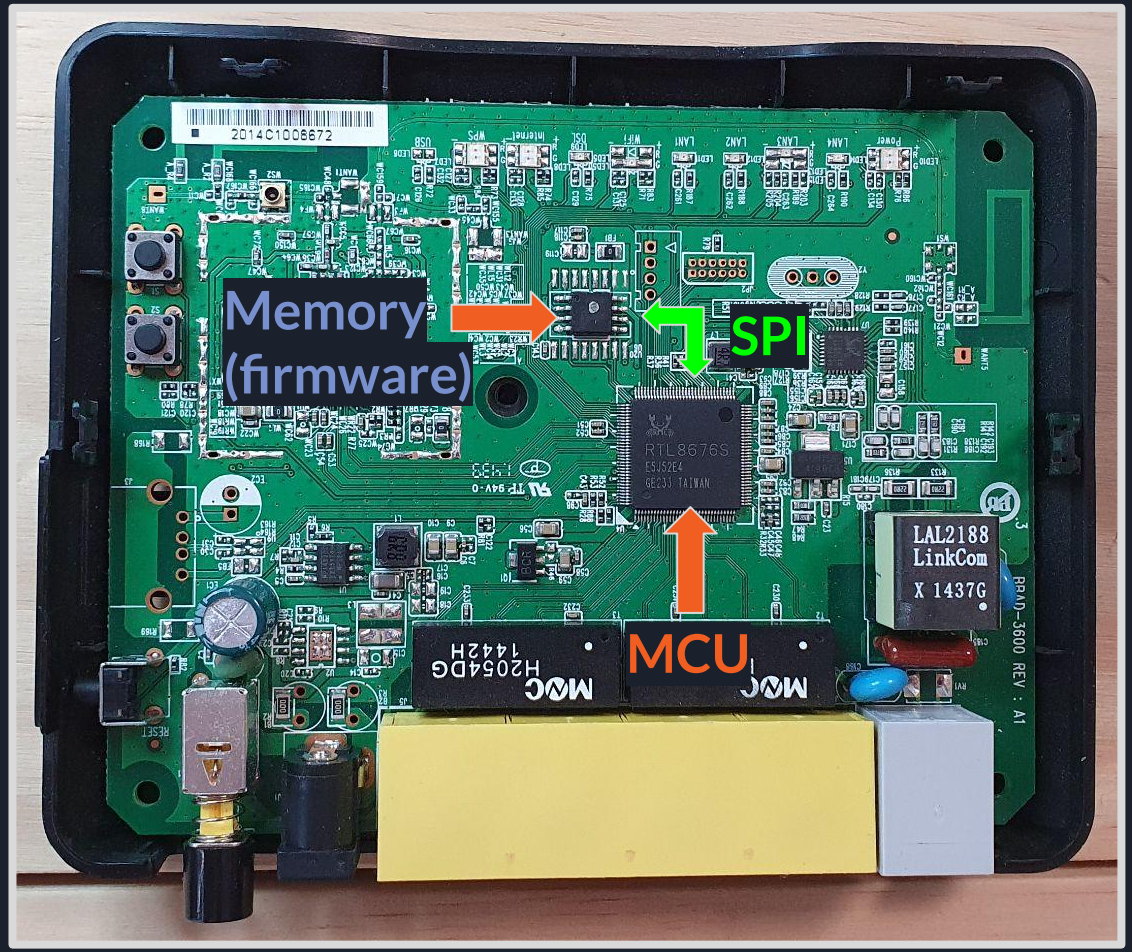
Logic Analyzer

- Device that captures signals of a digital target. Wireshark for hardware.
- It samples (reads) a voltage on a given pin thousands times a second
- It’s important to set a sampling rate higher then a given protocol transmission rate, otherwise we would miss data
- We love Saleae logic analyzer



Logic Analyzer

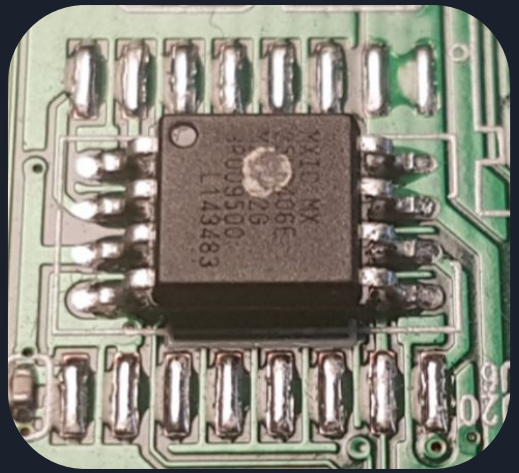




- MCU is the processing unit. RAM is built in the MCU chip.
  - SoC systems can also have built-in storage
- Router's firmware data is stored outside of the MCU and requires memory with large capacity.
- On Power-On, the MCU bootloader will fetch data from the NOR Flash using SPI Protocol.

Exercise
UART Bootlog Analysis

Flash Memory Types



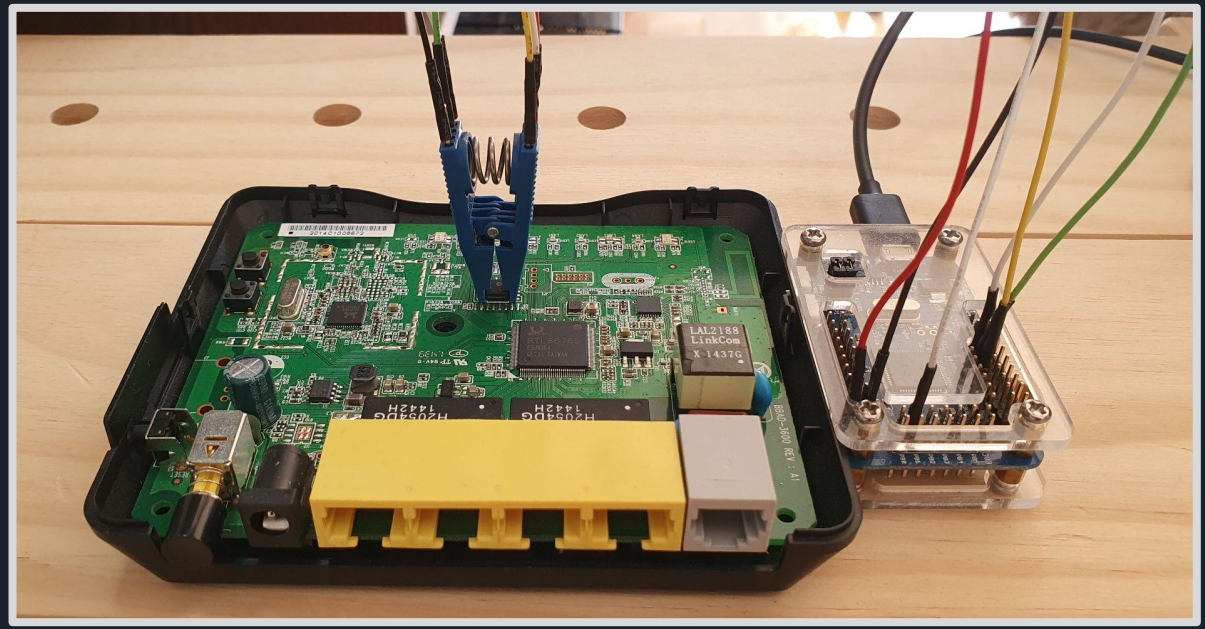
- NOR Flash
- SOIC8 package

- NAND Flash
- TSOP48 package

- eMMC Flash
- BGA{153} package

Physical Wiring

- Target is Powered-Off
- Master provide power to the Flash chip
- If target boots up wait some time after the boot-up process is finished. Otherwise you might interfere with a legitimate communication.

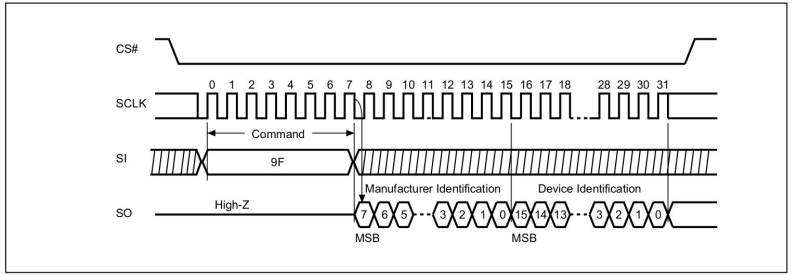


SPI Read-ID

- CS: By pulling down a target chip is selected
- SCLK: Master provides clock signal
- MOSI: Send hex 0x9F command to Read-ID
- MISO: Read data

MXIC
Macronix International Co., Ltd.
MX25L6406E

Figure 26. Read Identification (RDID) Sequence (Command 9F)

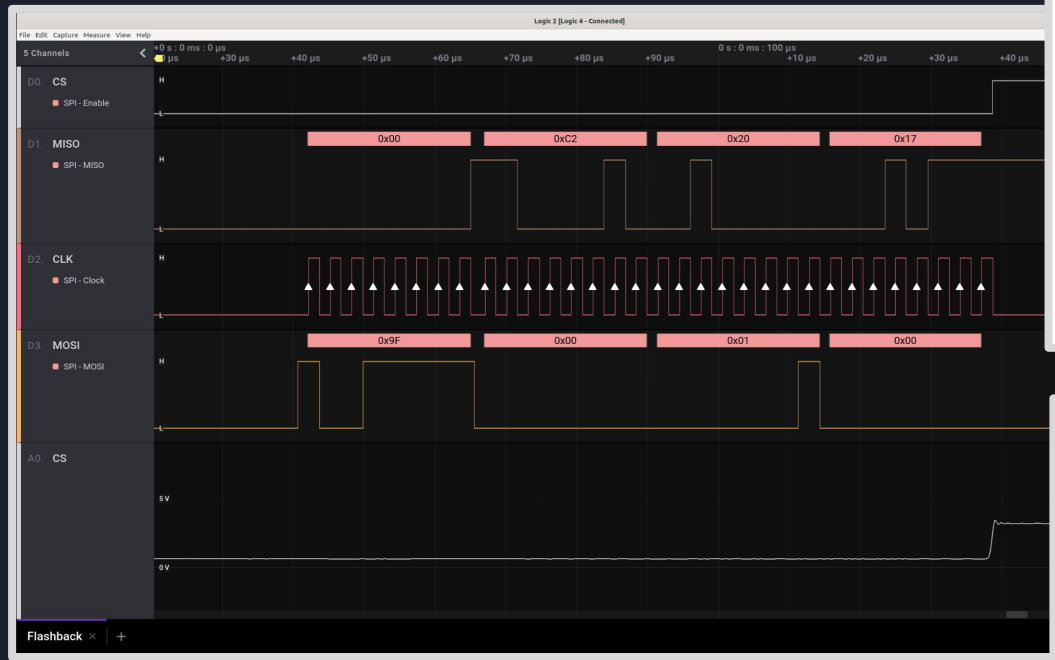


MXIC
Macronix International Co., Ltd.
MX25L6406E

Table 6. ID Definition

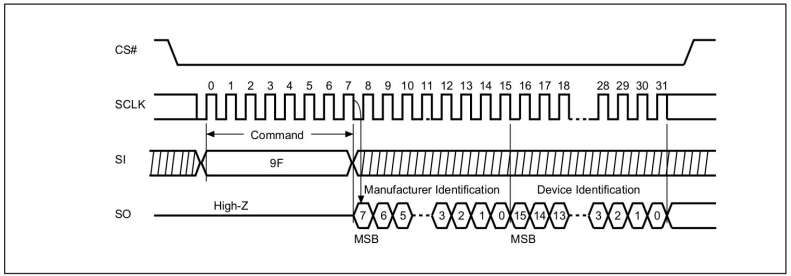
Command Type	MX25L6406E		
RDID Command	manufacturer ID	memory type	memory density
	C2	20	17
RES Command	electronic ID		
	16		
REMS Command	manufacturer ID	device ID	
	C2	16	

SPI Read-ID on a wire



MXIC
Macronix International Co., Ltd.
MX25L6406E

Figure 26. Read Identification (RDID) Sequence (Command 9F)



MXIC
Macronix International Co., Ltd.
MX25L6406E

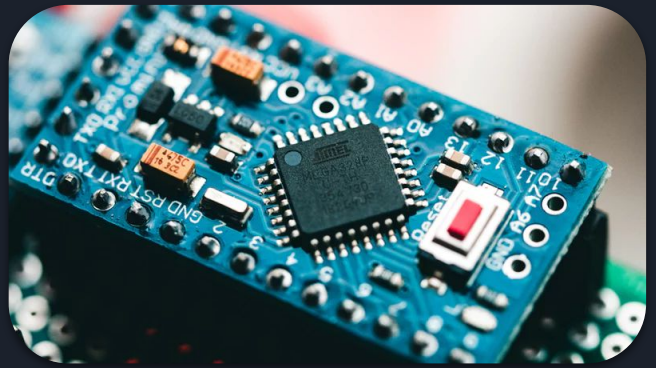
Table 6. ID Definition

Command Type	MX25L6406E		
RDID Command	manufacturer ID	memory type	memory density
	C2	20	17
RES Command	electronic ID		
	16		
REMS Command	manufacturer ID	device ID	
	C2	16	

Exercise
NOR Flash

Microcontrollers

Microcontrollers



- Small built-in RAM and ROM
- In many cases no operating system or small RTOS
- Real-time interrupts: when a certain event occurs Microcontrollers pauses code execution, invokes an interrupt handler and only after returns to paused code execution.
- Memory mapped I/O: the same address space for memory and I/O devices
  - It means that when the program is performing read or write on the memory address it may be equal to read/write to the physical peripheral.

Microcontroller Vector Table

- Special data structure called Vector Table. Usually located at the start of the memory at 0x0000.0000 address.
- Its purpose is to manage exceptions handling.
  - Reset is one of such an exception. Whenever the microcontroller powers-up, the Reset exception is called. The address located at the Reset vector is the first instruction that the CPU will execute.
- Reset Handler is crucial for loading firmware into IDA/Ghidra

Figure 2-6. Vector Table

Exception number	IRQ number	Offset	Vector
154	138	0x0268	IRQ131
. . .		. . .	. . .
18	2	0x004C	IRQ2
17	1	0x0048	IRQ1
16	0	0x0044	IRQ0
15	-1	0x0040	Systick
14	-2	0x003C	PendSV
13		0x0038	Reserved
12			Reserved for Debug
11	-5		SVCall
10		0x002C	
9			Reserved
8			
7			
6	-10		Usage fault
5	-11	0x0018	Bus fault
4	-12	0x0014	Memory management fault
3	-13	0x0010	Hard fault
2	-14	0x000C	NMI
1		0x0008	Reset
		0x0004	Initial SP value
		0x0000	

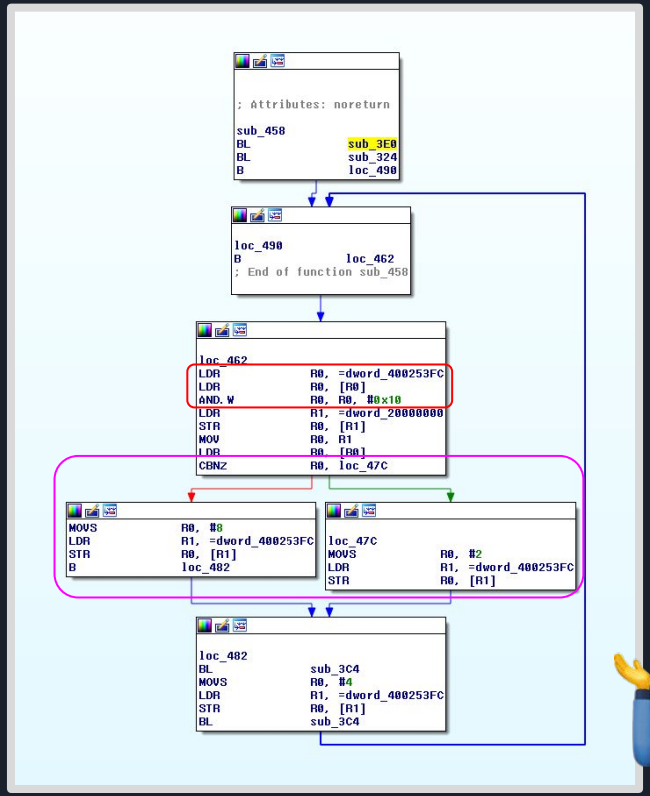
Texas Instruments TM4C123G

Main function

- 0x40025000 is the base of the GPIO Port F register
- offset 0x000 - 0x400 is described as “GPIO Data”
- 0x3FC = 0011.1111.1100
- AND operation to test bit4

Figure 10-4. GPIODATA Read Example

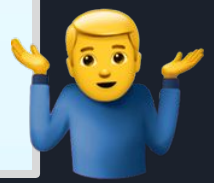
	9	8	7	6	5	4	3	2	1	0
ADDR[9:2]										
0x0C4	0	0	1	1	0	0	0	1	0	0
GPIODATA	1	0	1	1	1	1	1	0		
Returned Value	0	0	1	1	0	0	0	0		
	7	6	5	4	3	2	1	0		



- If Bit4 is 0 the program sets Port F to 0x2 else the PortF is set to 0x8.

Figure 10-3. GPIODATA Write Example

	9	8	7	6	5	4	3	2	1	0
ADDR[9:2]										
0x098	0	0	1	0	0	1	1	0	0	0
0xEB	1	1	1	0	1	0	1	1		
GPIODATA	u	u	1	u	u	0	1	u		
	7	6	5	4	3	2	1	0		

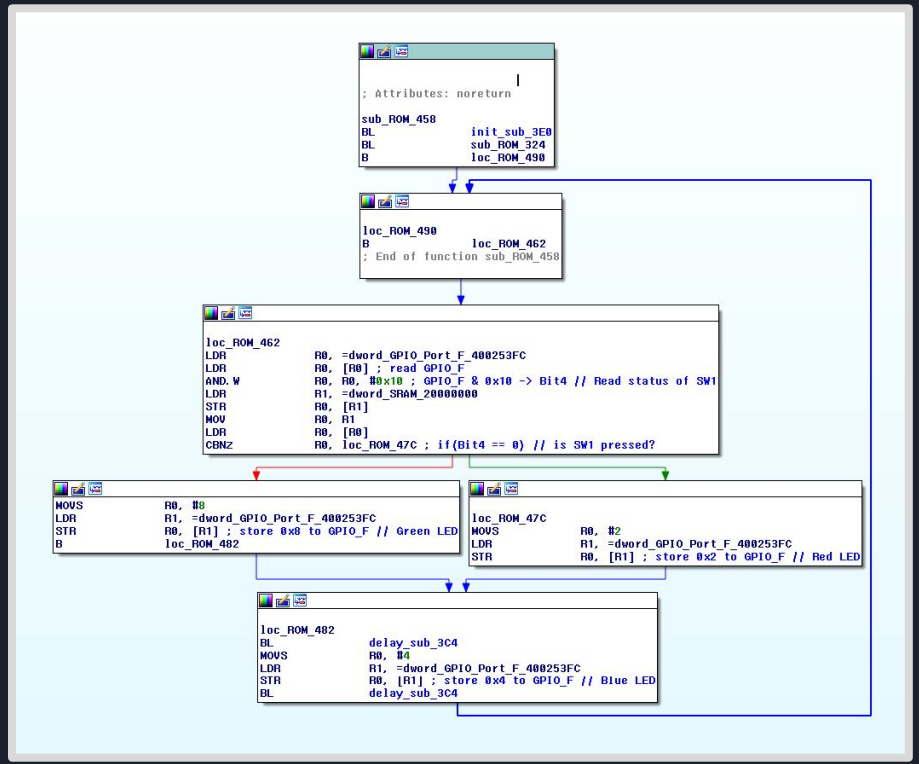


Apply some logic

- From a data sheet: GPIO Port F is connected to the RGB LED and 2 physical switches.

GPIO Pin	Pin Function	USB Device
PF4	GPIO	SW1
PF0	GPIO	SW2
PF1	GPIO	RGB LED (Red)
PF2	GPIO	RGB LED (Blue)
PF3	GPIO	RGD LED (Green)

- the program reads the status of Switch1 (Pressed = 0, Unpressed = 1).
- If the SW1 is pressed the LED is turned Red.
- Otherwise the LED is turned Green.
- After the time delay is elapsed the LED is turned Blue.
- The loop starts from the beginning.



Microcontroller Summary

- Reverse engineering of microcontroller firmware is a little bit different from reversing of ELF compiled binaries. IDA / Ghidra will not understand the binary data by default.
- It requires some knowledge how microcontroller works in order to recognize the special functionality in the assembly.
- Reading the chip's datasheet is crucial as chips differ in functionality and build from each other.
- Some vendors provide SVD files with a memory layout. Think about it as "symbols" for memory mapped IO.
  - Use SVD Loader script for ghidra. Kudos to @ghidraninja

RTOS

RTOS (Real Time Operating System)

- RTOS are “bare metal” operating systems
  - Perform CPU and peripheral initialisation themselves
  - RTOS code is usually a single monolithic binary, not split into separate files
    - No kernel and userland; everything is “kernel”

- What does Real Time mean?
  - Guaranteed execution within a certain number of milliseconds
  - RTOS are used for anything that requires strict time observance
    - From your phone’s baseband to space ships
    - If you are landing your space ship, you need a certain thing to happen within a certain time frame…
    - “Normal” operating systems (Windows, MacOS, Linux, etc) do not have these guarantees

RTOS are everywhere

A&D	Telecom	Transportation	Medical	Manufacturing
• Flight display controller • Engine turbine • Drones • Extraterrestrial rovers	• 5G modem • Satellite modem • Base station	• Functional safety systems • Emergency braking systems • Engine warning systems	• Magnetic resonance imaging • Surgery equipment • Ventilators	• Factory robotics systems • Safety systems • Oil and gas vibration monitors

Examples: FreeRTOS, e-Cos, VxWorks, QNX, Zephyr, Green Hills, SMX, Nucleus, and the list goes on and on!

RTOS - Internals

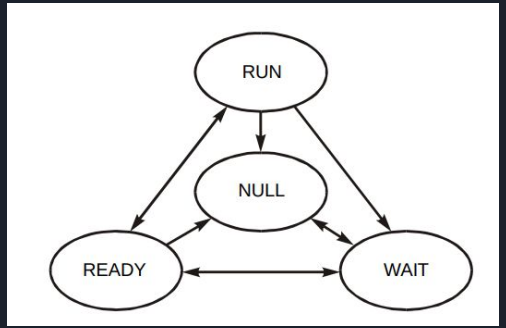
- Is not split into separate binaries like a high-level OS. Usually there is
  - some startup code,
  - some library routines,
  - user-provided code in forms of tasks

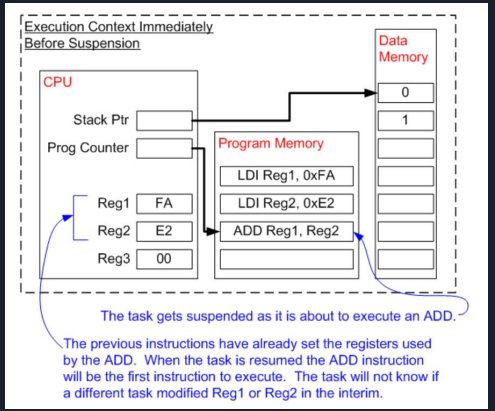
- Tasks are nothing more than simple functions performing the necessary work in a simple infinite loop.

- The “main function” called by the RTOS startup would:
  - register the tasks,
  - set up the shared resources like timers, queues, semaphores etc.
  - invoke the RTOS entry point which starts dispatching the tasks, switching between them periodically so that each gets a chance to run.

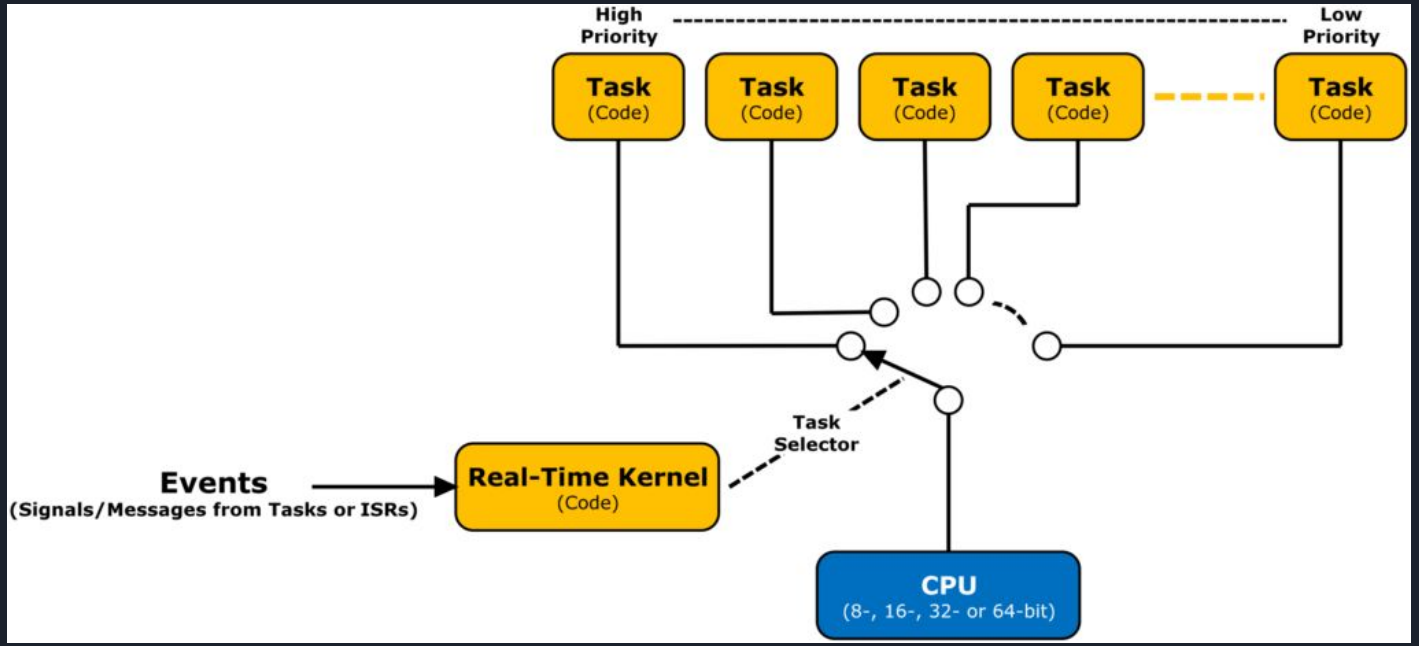
Tasks

- A code performing a specific function
- Application can have many tasks but only 1 task can run at a time on a single cored processor
- Task has its own dedicated stack
- Switching from Task A to Task B is called context switching and require Kernel to save CPU register state and stack state
- Tasks are invoked or “woken up” when a certain event happens
  - Interrupt
  - Timer (X time has passed)
  - Important message received





RTOS - Tasks


High Priority
Low Priority
Task (Code)
Task (Code)
Task (Code)
Task (Code)
Task (Code)
Task Selector
Events
(Signals/Messages from Tasks or ISRs)
Real-Time Kernel
(Code)
CPU
(8-, 16-, 32- or 64-bit)

Task’s life @FreeRTOS

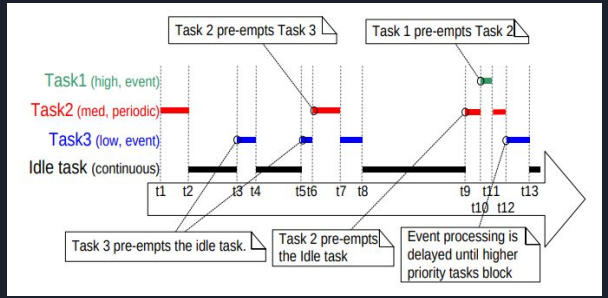
```
QueueHandle_t mailbox;

void task1(void* p)
{
       uint32_t data = 1337;
    while(1) {
        puts("Hello Flashback!");
        xQueueSend(mailbox, &data, 0);
        vTaskDelay(1000);
    }
}

void task2(void* p)
{
       uint32_t data = 0;
    while(1) {
       xQueueReceive(mailbox, &data, 0);
        puts("Hello %d Hackers!", data);
        vTaskDelay(1000);
    }
}

int main()
{
    xTaskCreate(task1, (signed char*)"FlashbackTask", STACK_BYTES(2048), 0, 1, 0);
    xTaskCreate(task2, (signed char*)"WorldTask", STACK_BYTES(2048), 0, 1, 0);
    vTaskStartScheduler();

    for( ;; );
}
```

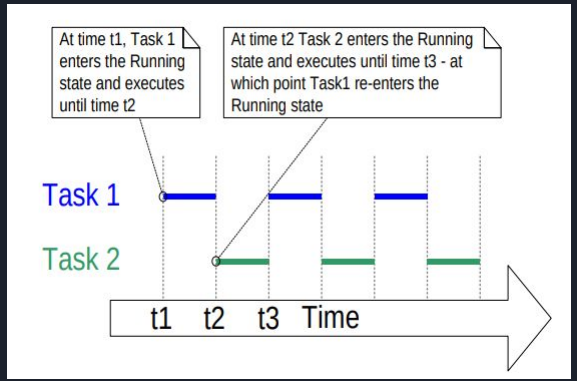
Task Schedulers

- Preemptive scheduling algorithms
  - Allows interfuption of a running Task if task with a higher priority requests execution time

- Non-preemptive scheduling algorithms
  - Scheduler can only start a task and then it has to wait for the task to finish or for the task to voluntarily return the control.

  - A running task can’t be stopped by the scheduler.





Inter-Task communication

- Tasks need to communicate with each other!
- In a normal OS (non RTOS) can use pipes, message queues, Remote Procedure Calls, Sockets, etc, to pass messages between processes
- In RTOS to pass messages between tasks we can use:
  - Shared Variables or Memory Areas
  - Queues - put a message on a queue to some other task
  - Mailboxes - like a queue but a message has a predefined structure
  - Signals - tell some other tasks to do something or to synchronize
  - Mutex / Semaphore - synchronize access to a shared resource, lock resource which is in-use
- Good video series about RTOS internals and programming: https://www.youtube.com/watch?v=F321087yYy4

Reset Vector Table for Cortex R and A

- The previously shown Reset Vector Table (RVT) was for Cortex M (microcontroller)
- Cortex R and A processors have a lot more features, so RVT is going to be more complex!
- Check ARM documentation:

  https://developer.arm.com/documentation/den0042/a/Boot-Code/Booting-a-bare-metal-system

RTOS Exercise ☠️